

Comprehensive Causal Inference Analysis: Policy Effectiveness Evaluation

Author: Aadip Thapaliya | **Course:** Causal Inference and Machine Learning | **Date:** June 2026

Project Overview

This notebook presents a **comprehensive causal inference analysis** evaluating the effectiveness of five distinct government policy interventions across different economic and social domains. The analysis applies state-of-the-art causal machine learning methods to estimate policy effects while rigorously addressing confounding bias.

Policy Domains Analyzed

#	Policy Domain	Treatment Variable	Outcome Variable	Key Confounders
1	Job Training	Training Program (0/1)	Employment Growth	Labor Market Volatility, Baseline Employment, Employer Confidence
2	Tax Incentive	Tax Incentive (0/1)	Sales Growth	Consumer Demand Index, Baseline Retail Sales, Consumer Spending
3	Economic Stimulus	Stimulus Policy (0/1)	Employment Rate	Consumer Confidence, Market Volatility, Baseline Economic Growth
4	Education Subsidy	Subsidy Policy (0/1)	Graduation Rate	Education Quality Index, Baseline Graduation Rate, School Enrollment
5	Infrastructure	Infrastructure Spending (0/1)	Investment Return	Construction Cost Index, Baseline Investment Rate, Construction Activity

Methods Applied

- **Propensity Score Estimation & IPW** — Balance assessment and inverse probability weighting
- **Doubly Robust Estimation (AIPW)** — Augmented inverse probability weighting for robust ATE
- **Meta-Learners** — S-Learner and T-Learner for comparison
- **Causal Forests** — Heterogeneous treatment effect estimation with honest trees
- **Sensitivity Analysis** — Rosenbaum bounds for robustness to unmeasured confounding
- **Placebo Tests** — Irrelevant variable validation
- **DAG-based Identification** — Causal structure visualization

In []:

```
# =====  
# CELL 1: IMPORTS AND CONFIGURATION  
# =====  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
import networkx as nx  
from scipy import stats  
from sklearn.linear_model import LogisticRegression, LinearRegression  
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor  
from sklearn.preprocessing import StandardScaler  
from sklearn.model_selection import train_test_split  
import warnings  
warnings.filterwarnings('ignore')  
  
# Visualization style  
plt.style.use('seaborn-v0_8-whitegrid')  
sns.set_palette("husl")  
  
print(" All packages imported successfully!")  
print(" Ready for comprehensive causal inference analysis...")
```

All packages imported successfully!
Ready for comprehensive causal inference analysis...

Section 1: Data Loading & Characterization

We begin by loading all five policy datasets and examining their structure, treatment balance, and outcome distributions.

```
In [ ]: # =====
# CELL 2: LOAD ALL DATASETS
# =====

# Load all policy datasets
datasets = {
    'Job Training': pd.read_csv('job_training_policy.csv'),
    'Tax Incentive': pd.read_csv('tax_incentive_policy.csv'),
    'Economic Stimulus': pd.read_csv('economic_stimulus.csv'),
    'Education Subsidy': pd.read_csv('education_subsidy_policy.csv'),
    'Infrastructure': pd.read_csv('infrastructure_spending_policy.csv')
}

# Standardized configuration for each policy domain
policy_configs = {
    'Job Training': {
        'df': datasets['Job Training'],
        'treatment': 'training_program',
        'outcome': 'employment_growth',
        'confounders': ['labor_market_volatility', 'baseline_employment_trend', 'employer_confidence'],
        'irrelevant': 'irrelevant_index',
        'label': 'Job Training Policy'
    },
    'Tax Incentive': {
        'df': datasets['Tax Incentive'],
        'treatment': 'tax_incentive',
        'outcome': 'sales_growth',
        'confounders': ['consumer_demand_index', 'baseline_retail_sales', 'consumer_spending'],
        'irrelevant': 'irrelevant_index',
        'label': 'Tax Incentive Policy'
    },
    'Economic Stimulus': {
        'df': datasets['Economic Stimulus'],
        'treatment': 'stimulus_policy',
        'outcome': 'employment_rate',
        'confounders': ['consumer_confidence', 'market_volatility', 'baseline_economic_growth'],
        'irrelevant': 'unrelated_indicator',
        'label': 'Economic Stimulus Policy'
    },
    'Education Subsidy': {
        'df': datasets['Education Subsidy'],
        'treatment': 'subsidy_policy',
        'outcome': 'graduation_rate',
        'confounders': ['education_quality_index', 'baseline_graduation_rate', 'school_enrollment'],
        'irrelevant': 'irrelevant_index',
        'label': 'Education Subsidy Policy'
    },
    'Infrastructure': {
        'df': datasets['Infrastructure'],
        'treatment': 'infrastructure_spend',
        'outcome': 'investment_return',
        'confounders': ['construction_cost_index', 'baseline_investment_rate', 'construction_activity'],
        'irrelevant': 'irrelevant_index',
        'label': 'Infrastructure Spending Policy'
    }
}

# Display dataset information
print("="*80)
print("DATASET OVERVIEW")
print("="*80)
for name, config in policy_configs.items():
    df = config['df']
    treatment = config['treatment']
    print(f"\n {name}:")
    print(f"   Shape: {df.shape[0]} rows x {df.shape[1]} columns")
    print(f"   Treatment: {treatment} (Balance: {df[treatment].value_counts().to_dict()})")
    print(f"   Outcome: {config['outcome']}")
    print(f"   Confounders: {config['confounders']}")
    print(f"   Missing Values: {df.isnull().sum().sum()}")
```

DATASET OVERVIEW

Job Training:

Shape: 500 rows × 6 columns
Treatment: training_program (Balance: {1: 276, 0: 224})
Outcome: employment_growth
Confounders: ['labor_market_volatility', 'baseline_employment_trend', 'employer_confidence']
Missing Values: 0

Tax Incentive:

Shape: 500 rows × 6 columns
Treatment: tax_incentive (Balance: {0: 254, 1: 246})
Outcome: sales_growth
Confounders: ['consumer_demand_index', 'baseline_retail_sales', 'consumer_spending']
Missing Values: 0

Economic Stimulus:

Shape: 500 rows × 6 columns
Treatment: stimulus_policy (Balance: {1: 254, 0: 246})
Outcome: employment_rate
Confounders: ['consumer_confidence', 'market_volatility', 'baseline_economic_growth']
Missing Values: 0

Education Subsidy:

Shape: 500 rows × 6 columns
Treatment: subsidy_policy (Balance: {1: 256, 0: 244})
Outcome: graduation_rate
Confounders: ['education_quality_index', 'baseline_graduation_rate', 'school_enrollment']
Missing Values: 0

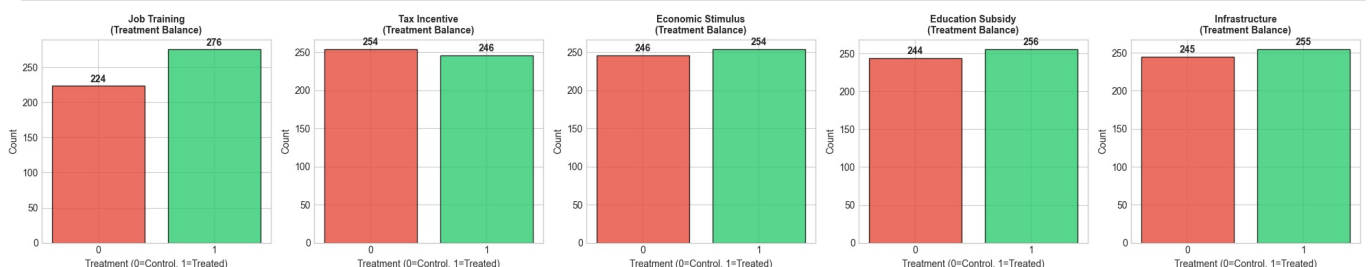
Infrastructure:

Shape: 500 rows × 6 columns
Treatment: infrastructure_spend (Balance: {1: 255, 0: 245})
Outcome: investment_return
Confounders: ['construction_cost_index', 'baseline_investment_rate', 'construction_activity']
Missing Values: 0

1.1 Treatment Balance Visualization

Visualizing the distribution of treatment assignment across all five policy domains.

```
In [ ]: # =====  
# CELL 3: TREATMENT BALANCE VISUALIZATION  
# =====  
  
fig, axes = plt.subplots(1, 5, figsize=(20, 4))  
  
for idx, (name, config) in enumerate(policy_configs.items()):  
    df = config['df']  
    treatment_col = config['treatment']  
    counts = df[treatment_col].value_counts().sort_index()  
  
    bars = axes[idx].bar(counts.index.astype(str), counts.values,  
                          color=['#e74c3c', '#2ecc71'], alpha=0.8, edgecolor='black')  
    axes[idx].set_title(f'{name}\n(Treatment Balance)', fontsize=10, fontweight='bold')  
    axes[idx].set_xlabel('Treatment (0=Control, 1=Treated)')  
    axes[idx].set_ylabel('Count')  
  
    for i, v in enumerate(counts.values):  
        axes[idx].text(i, v + 5, str(v), ha='center', fontweight='bold')  
  
plt.tight_layout()  
plt.savefig('treatment_balance.png', dpi=150, bbox_inches='tight')  
plt.show()  
print(" Treatment balance visualization complete!")
```



Treatment balance visualization complete!

1.2 Outcome Distribution by Treatment Group

Examining the raw outcome differences between treated and control groups before any causal adjustment.

```
In [ ]: # =====
# CELL 4: OUTCOME COMPARISON (RAW)
# =====

fig, axes = plt.subplots(2, 5, figsize=(22, 10))
raw_diffs = {}

for idx, (name, config) in enumerate(policy_configs.items()):
    df = config['df']
    treatment = df[config['treatment']]
    outcome = df[config['outcome']]

    treated_outcome = outcome[treatment == 1]
    control_outcome = outcome[treatment == 0]

    # Distribution plot
    axes[0, idx].hist(control_outcome, bins=25, alpha=0.6, label='Control',
                      color='#e74c3c', edgecolor='black')
    axes[0, idx].hist(treated_outcome, bins=25, alpha=0.6, label='Treated',
                      color='#2ecc71', edgecolor='black')
    axes[0, idx].set_title(f'{name}\nOutcome Distribution', fontweight='bold')
    axes[0, idx].legend()

    # Box plot
    bp = axes[1, idx].boxplot([control_outcome, treated_outcome],
                              tick_labels=['Control', 'Treated'], patch_artist=True) # ← fixed
    bp['boxes'][0].set_facecolor('#e74c3c')
    bp['boxes'][1].set_facecolor('#2ecc71')
    axes[1, idx].set_title('Control vs Treated', fontweight='bold')
    axes[1, idx].grid(axis='y', alpha=0.3)

    # Statistics
    t_stat, p_val = stats.ttest_ind(treated_outcome, control_outcome)
    diff = treated_outcome.mean() - control_outcome.mean()
    raw_diffs[name] = diff

    print(f"\n{name}:")
    print(f"  Control: {control_outcome.mean():.1f} (±{control_outcome.std():.1f})")
    print(f"  Treated: {treated_outcome.mean():.1f} (±{treated_outcome.std():.1f})")
    print(f"  Raw Difference: {diff:.1f} (p={p_val:.2e})")

plt.tight_layout()
plt.savefig('outcome_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```

Job Training:

Control: 117.1 (±4.7)
Treated: 220.3 (±9.9)
Raw Difference: 103.2 (p=0.00e+00)

Tax Incentive:

Control: 117.7 (±4.5)
Treated: 231.9 (±11.8)
Raw Difference: 114.2 (p=0.00e+00)

Economic Stimulus:

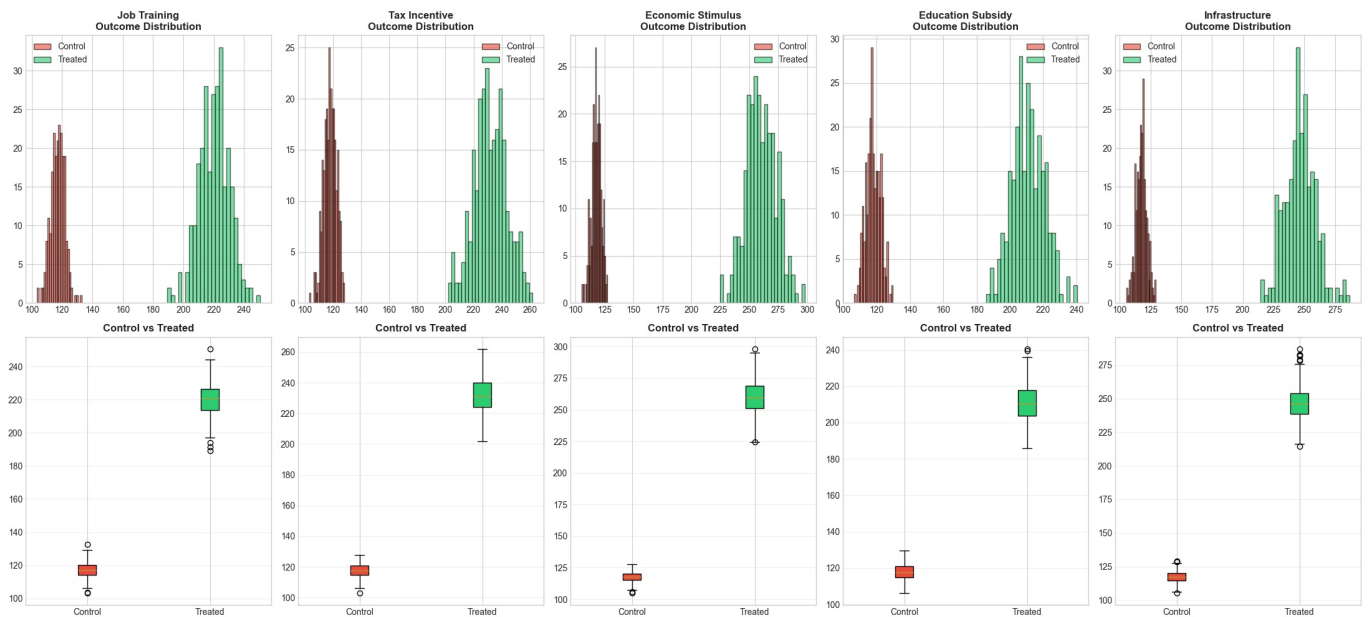
Control: 117.8 (±4.1)
Treated: 260.4 (±13.2)
Raw Difference: 142.6 (p=0.00e+00)

Education Subsidy:

Control: 117.9 (±4.4)
Treated: 211.0 (±9.9)
Raw Difference: 93.1 (p=0.00e+00)

Infrastructure:

Control: 117.5 (±4.3)
Treated: 246.5 (±12.6)
Raw Difference: 129.0 (p=0.00e+00)



Section 2: Causal DAGs — Identification Strategy

Directed Acyclic Graphs (DAGs) formalize our causal assumptions. Each policy follows the same identification structure: confounders affect both treatment assignment and the outcome, requiring adjustment to identify the causal effect.

```
In [ ]: # =====
# CELL 5: CAUSAL DAG VISUALIZATION
# =====

fig, axes = plt.subplots(1, 5, figsize=(28, 5))

for idx, (name, config) in enumerate(policy_configs.items()):
    G = nx.DiGraph()

    treatment = config['treatment']
    outcome = config['outcome']
    confounders = config['confounders']

    # Add nodes
    G.add_node('T', label=treatment.replace('_', '\n'))
    G.add_node('Y', label=outcome.replace('_', '\n'))
    for i, c in enumerate(confounders):
        G.add_node(f'C{i}', label=c.replace('_', '\n'))
    G.add_node('U', label='Unobserved\nConfounders')

    # Add edges
    for i in range(len(confounders)):
        G.add_edge(f'C{i}', 'T')
        G.add_edge(f'C{i}', 'Y')
    G.add_edge('T', 'Y')
    G.add_edge('U', 'T')
    G.add_edge('U', 'Y')

    # Layout
    pos = {'T': (0, 0), 'Y': (2, 0), 'U': (1, 1.5)}
    for i in range(len(confounders)):
        pos[f'C{i}'] = (0.5 + i*0.5, -1.2)

    node_colors = ['#3498db', '#e74c3c'] + ['#2ecc71']*len(confounders) + ['#95a5a6']

    nx.draw(G, pos, ax=axes[idx], with_labels=True, node_color=node_colors,
            node_size=2500, font_size=7, font_weight='bold', edge_color='#34495e',
```

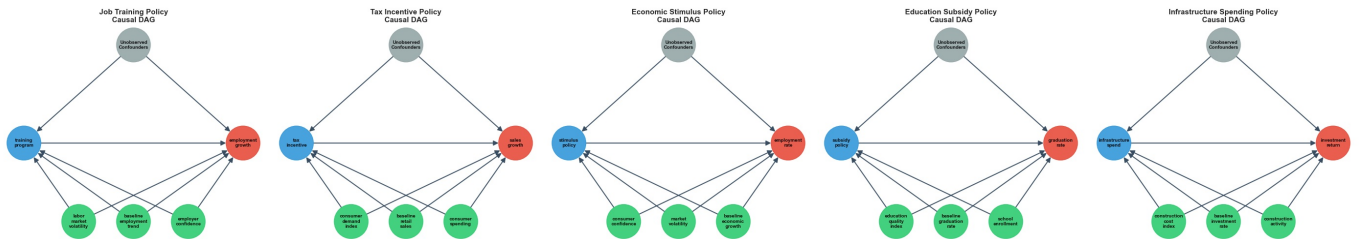
```

arrows=True, arrowsize=15, width=1.5, alpha=0.9,
labels={n: G.nodes[n]['label'] for n in G.nodes()})

axes[idx].set_title(f"{config['label']}\nCausal DAG", fontweight='bold', fontsize=11)
axes[idx].set_axis_off()

plt.tight_layout()
plt.savefig('causal_dags.png', dpi=150, bbox_inches='tight')
plt.show()
print(" Causal DAGs visualized!")

```



Causal DAGs visualized!

⚖️ Section 3: Propensity Score Analysis & Balance Assessment

Propensity scores model the probability of receiving treatment given observed confounders. We use logistic regression to estimate these scores and assess covariate balance using Standardized Mean Differences (SMD) before and after Inverse Probability Weighting (IPW).

```

In [ ]: # =====
# CELL 6: PROPENSITY SCORE ESTIMATION & BALANCE
# =====

def estimate_propensity_scores(df, treatment_col, confounder_cols):
    """Estimate propensity scores using logistic regression"""
    X = df[confounder_cols].values
    y = df[treatment_col].values
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
    ps_model = LogisticRegression(max_iter=1000, random_state=42)
    ps_model.fit(X_scaled, y)
    propensity_scores = ps_model.predict_proba(X_scaled)[:, 1]
    return propensity_scores, ps_model

def calculate_smd(df, treatment_col, confounder_cols, weights=None):
    """Calculate Standardized Mean Differences"""
    smds = {}
    for col in confounders:
        treated_mask = df[treatment_col].values == 1
        control_mask = ~treated_mask

        if weights is not None:
            treated_mean = np.average(df.loc[treated_mask, col], weights=weights[treated_mask])
            control_mean = np.average(df.loc[control_mask, col], weights=weights[control_mask])
            treated_var = np.average((df.loc[treated_mask, col]-treated_mean)**2, weights=weights[treated_mask])
            control_var = np.average((df.loc[control_mask, col]-control_mean)**2, weights=weights[control_mask])
        else:
            treated_mean = df.loc[treated_mask, col].mean()
            control_mean = df.loc[control_mask, col].mean()
            treated_var = df.loc[treated_mask, col].var()
            control_var = df.loc[control_mask, col].var()

        pooled_std = np.sqrt((treated_var + control_var) / 2)
        smds[col] = abs(treated_mean - control_mean) / pooled_std
    return smds

ps_results = {}

fig, axes = plt.subplots(2, 5, figsize=(24, 10))

for idx, (name, config) in enumerate(policy_configs.items()):
    df = config['df'].copy()
    treatment = config['treatment']
    confounders = config['confounders']

    # Estimate propensity scores
    ps, ps_model = estimate_propensity_scores(df, treatment, confounders)
    df['propensity_score'] = ps

    # Distribution plot

```

```

treated_ps = ps[df[treatment] == 1]
control_ps = ps[df[treatment] == 0]
axes[0, idx].hist(control_ps, bins=25, alpha=0.6, label='Control',
                  color='#e74c3c', density=True, edgecolor='black')
axes[0, idx].hist(treated_ps, bins=25, alpha=0.6, label='Treated',
                  color='#2ecc71', density=True, edgecolor='black')
axes[0, idx].set_title(f'{name}\nPropensity Score', fontweight='bold')
axes[0, idx].legend()

# IPW weights
p_t = df[treatment].mean()
df['stabilized_weight'] = np.where(df[treatment] == 1,
                                   p_t / df['propensity_score'],
                                   (1-p_t) / (1-df['propensity_score']))

# SMD comparison
smd_before = calculate_smd(df, treatment, confounders)
smd_after = calculate_smd(df, treatment, confounders, weights=df['stabilized_weight'].values)

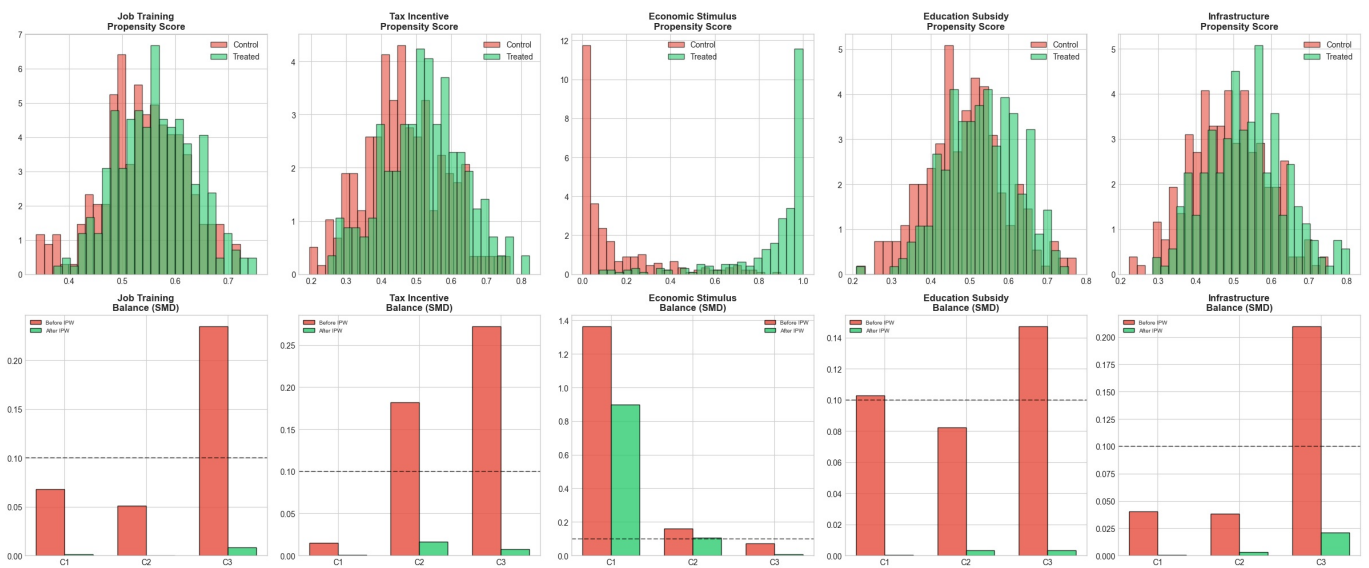
x = np.arange(len(confounders))
width = 0.35
before_vals = [smd_before[c] for c in confounders]
after_vals = [smd_after[c] for c in confounders]

axes[1, idx].bar(x - width/2, before_vals, width, label='Before IPW',
                 color='#e74c3c', alpha=0.8, edgecolor='black')
axes[1, idx].bar(x + width/2, after_vals, width, label='After IPW',
                 color='#2ecc71', alpha=0.8, edgecolor='black')
axes[1, idx].axhline(y=0.1, color='black', linestyle='--', alpha=0.5)
axes[1, idx].set_title(f'{name}\nBalance (SMD)', fontweight='bold')
axes[1, idx].set_xticks(x)
axes[1, idx].set_xticklabels([f'C{i+1}' for i in range(len(confounders))])
axes[1, idx].legend(fontsize=7)

ps_results[name] = {'df': df, 'smd_before': smd_before, 'smd_after': smd_after}

plt.tight_layout()
plt.savefig('propensity_scores.png', dpi=150, bbox_inches='tight')
plt.show()
print(" Propensity score analysis complete!")

```



Propensity score analysis complete!

3.1 Love Plot — Balance Assessment

Love plots visualize the standardized mean differences before and after weighting, showing how well IPW achieves covariate balance.

```

In [ ]: # =====
# CELL 7: LOVE PLOTS FOR BALANCE
# =====

fig, axes = plt.subplots(1, 5, figsize=(24, 6))

for idx, (name, config) in enumerate(policy_configs.items()):
    res = ps_results[name]
    confounders = config['confounders']
    conf_short = [c.replace('_', '\n').title()[:20] for c in confounders]

    before_vals = [res['smd_before'][c] for c in confounders]
    after_vals = [res['smd_after'][c] for c in confounders]

```

```

y_pos = np.arange(len(confounders))

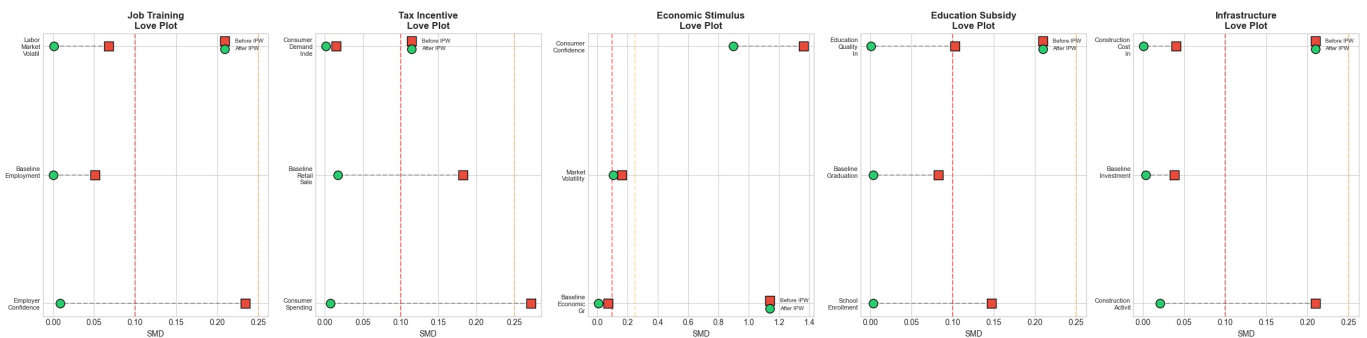
axes[idx].scatter(before_vals, y_pos, s=120, color='#e74c3c',
                  marker='s', label='Before IPW', zorder=5, edgecolors='black')
axes[idx].scatter(after_vals, y_pos, s=120, color='#2ecc71',
                  marker='o', label='After IPW', zorder=5, edgecolors='black')

for i in range(len(confounders)):
    axes[idx].plot([before_vals[i], after_vals[i]], [y_pos[i], y_pos[i]],
                  'k--', alpha=0.3)

axes[idx].axvline(x=0.1, color='red', linestyle='--', alpha=0.5)
axes[idx].axvline(x=0.25, color='orange', linestyle='--', alpha=0.3)
axes[idx].set_yticks(y_pos)
axes[idx].set_yticklabels(conf_short, fontsize=8)
axes[idx].set_xlabel('SMD')
axes[idx].set_title(f'{name}\nLove Plot', fontweight='bold')
axes[idx].legend(fontsize=7)
axes[idx].invert_yaxis()

plt.tight_layout()
plt.savefig('love_plots.png', dpi=150, bbox_inches='tight')
plt.show()

```



Section 4: Causal Effect Estimation — Multiple Methods

We apply five complementary estimation strategies to ensure robustness and validate our findings through method triangulation.

4.1 Method Definitions

Method	Type	Key Feature
Naive Diff-in-Means	Baseline	Unadjusted comparison (biased if confounding exists)
IPW (Hajek)	Reweighting	Inverse probability weighting for balance
AIPW (Doubly Robust)	Semiparametric	Combines outcome modeling + IPW; robust to misspecification
S-Learner	Meta-Learner	Single model with treatment as feature
T-Learner	Meta-Learner	Separate models for treated and control

The **AIPW estimator** is our primary estimator due to its double robustness property: it remains consistent if either the propensity score model OR the outcome model is correctly specified.

```

In [ ]: # =====
# CELL 8: CAUSAL EFFECT ESTIMATION FUNCTIONS
# =====

def aipw_ate(df, treatment_col, outcome_col, confounder_cols, propensity_scores):
    """Augmented Inverse Probability Weighting (Doubly Robust)"""
    T = df[treatment_col].values
    Y = df[outcome_col].values
    X = df[confounder_cols].values
    ps = np.clip(propensity_scores, 0.05, 0.95)

    # Outcome models
    mu1_model = GradientBoostingRegressor(n_estimators=100, max_depth=3, random_state=42)
    mu1_model.fit(X[T==1], Y[T==1])
    mu1_pred = mu1_model.predict(X)

    mu0_model = GradientBoostingRegressor(n_estimators=100, max_depth=3, random_state=42)
    mu0_model.fit(X[T==0], Y[T==0])
    mu0_pred = mu0_model.predict(X)

```



```

# AIPW scores
aipw_scores = (mu1_pred - mu0_pred) + \
               T*(Y - mu1_pred)/ps - (1-T)*(Y - mu0_pred)/(1-ps)

ate = np.mean(aipw_scores)
se = np.std(aipw_scores) / np.sqrt(len(Y))

return {
    'ate': ate,
    'se': se,
    'ci_lower': ate - 1.96*se,
    'ci_upper': ate + 1.96*se
}

def s_learner_ate(df, treatment_col, outcome_col, confounder_cols):
    """S-Learner: Single model with treatment as feature"""
    X = df[confounder_cols + [treatment_col]].values
    Y = df[outcome_col].values
    model = GradientBoostingRegressor(n_estimators=100, max_depth=3, random_state=42)
    model.fit(X, Y)

    X_t1, X_t0 = X.copy(), X.copy()
    X_t1[:, -1], X_t0[:, -1] = 1, 0
    cate = model.predict(X_t1) - model.predict(X_t0)
    return {'ate': np.mean(cate), 'cate': cate}

def t_learner_ate(df, treatment_col, outcome_col, confounder_cols):
    """T-Learner: Two separate models"""
    X, Y, T = df[confounder_cols].values, df[outcome_col].values, df[treatment_col].values
    m1 = GradientBoostingRegressor(n_estimators=100, max_depth=3, random_state=42)
    m1.fit(X[T==1], Y[T==1])
    m0 = GradientBoostingRegressor(n_estimators=100, max_depth=3, random_state=42)
    m0.fit(X[T==0], Y[T==0])
    cate = m1.predict(X) - m0.predict(X)
    return {'ate': np.mean(cate), 'cate': cate}

def ipw_hajek(df, treatment_col, outcome_col, ps):
    """Hajek estimator (normalized IPW)"""
    T, Y = df[treatment_col].values, df[outcome_col].values
    ps = np.clip(ps, 0.05, 0.95)
    ate = (np.sum(T*Y/ps)/np.sum(T/ps)) - (np.sum((1-T)*Y/(1-ps))/np.sum((1-T)/(1-ps)))
    return {'ate': ate}

print(" Estimation functions defined!")

```

Estimation functions defined!

```

In [ ]: # =====
# CELL 9: RUN ALL ESTIMATORS
# =====

all_results = {}

print("="*80)
print("COMPREHENSIVE CAUSAL EFFECT ESTIMATION RESULTS")
print("="*80)

for name, config in policy_configs.items():
    df = config['df'].copy()
    treatment = config['treatment']
    outcome = config['outcome']
    confounders = config['confounders']
    ps = ps_results[name]['df']['propensity_score'].values

    print(f"\n{'-'*60}")
    print(f" {name.upper()}")
    print(f"{'-'*60}")

    # Naive
    treated = df[df[treatment]==1][outcome]
    control = df[df[treatment]==0][outcome]
    naive_ate = treated.mean() - control.mean()
    print(f" Naive ATE:          {naive_ate:>8.2f}")

    # IPW
    ipw_res = ipw_hajek(df, treatment, outcome, ps)
    print(f" IPW (Hajek):          {ipw_res['ate']:>8.2f}")

    # AIPW
    aipw_res = aipw_ate(df, treatment, outcome, confounders, ps)
    print(f" AIPW ATE:             {aipw_res['ate']:>8.2f}   95% CI: {aipw_res['ci_lower']:.2f} - {aipw_res['ci_upper']:.2f}")

    # S-Learner

```

```

s_res = s_learner_ate(df, treatment, outcome, confounders)
print(f" S-Learner:      {s_res['ate']:>8.2f}")

# T-Learner
t_res = t_learner_ate(df, treatment, outcome, confounders)
print(f" T-Learner:      {t_res['ate']:>8.2f}")

# Store results
all_results[name] = {
    'naive': naive_ate,
    'ipw': ipw_res['ate'],
    'aipw': aipw_res['ate'],
    'aipw_ci': (aipw_res['ci_lower'], aipw_res['ci_upper']),
    's_learner': s_res['ate'],
    't_learner': t_res['ate'],
    'cate_s': s_res['cate'],
    'cate_t': t_res['cate']
}

print(f"\n{'='*80}")
print(" All estimations complete!")

```

=====

COMPREHENSIVE CAUSAL EFFECT ESTIMATION RESULTS

=====

JOB TRAINING

Naive ATE:	103.23	
IPW (Hajek):	101.94	
AIPW ATE:	101.80	[95% CI: 101.11 – 102.49]
S-Learner:	101.69	
T-Learner:	101.81	

TAX INCENTIVE

Naive ATE:	114.18	
IPW (Hajek):	112.34	
AIPW ATE:	112.39	[95% CI: 111.57 – 113.22]
S-Learner:	112.18	
T-Learner:	112.40	

ECONOMIC STIMULUS

Naive ATE:	142.60	
IPW (Hajek):	139.32	
AIPW ATE:	133.81	[95% CI: 132.83 – 134.79]
S-Learner:	133.25	
T-Learner:	133.75	

EDUCATION SUBSIDY

Naive ATE:	93.11	
IPW (Hajek):	92.53	
AIPW ATE:	92.51	[95% CI: 91.82 – 93.20]
S-Learner:	92.35	
T-Learner:	92.51	

INFRASTRUCTURE

Naive ATE:	128.99	
IPW (Hajek):	127.60	
AIPW ATE:	127.39	[95% CI: 126.50 – 128.27]
S-Learner:	127.23	
T-Learner:	127.40	

=====

All estimations complete!

4.2 ATE Comparison Visualization

Visualizing the convergence of all five estimation methods across policy domains.

```

In [ ]: # =====
# CELL 10: ATE COMPARISON VISUALIZATION
# =====

methods = ['Naive', 'IPW', 'AIPW', 'S-Learner', 'T-Learner']

```

```

policies = list(policy_configs.keys())

fig, ax = plt.subplots(figsize=(14, 8))
x = np.arange(len(policies))
width = 0.15
colors = ['#e74c3c', '#3498db', '#2ecc71', '#9b59b6', '#f39c12']

for i, method in enumerate(methods):
    values = []
    for policy in policies:
        if method == 'Naive': values.append(all_results[policy]['naive'])
        elif method == 'IPW': values.append(all_results[policy]['ipw'])
        elif method == 'AIPW': values.append(all_results[policy]['aipw'])
        elif method == 'S-Learner': values.append(all_results[policy]['s_learner'])
        elif method == 'T-Learner': values.append(all_results[policy]['t_learner'])

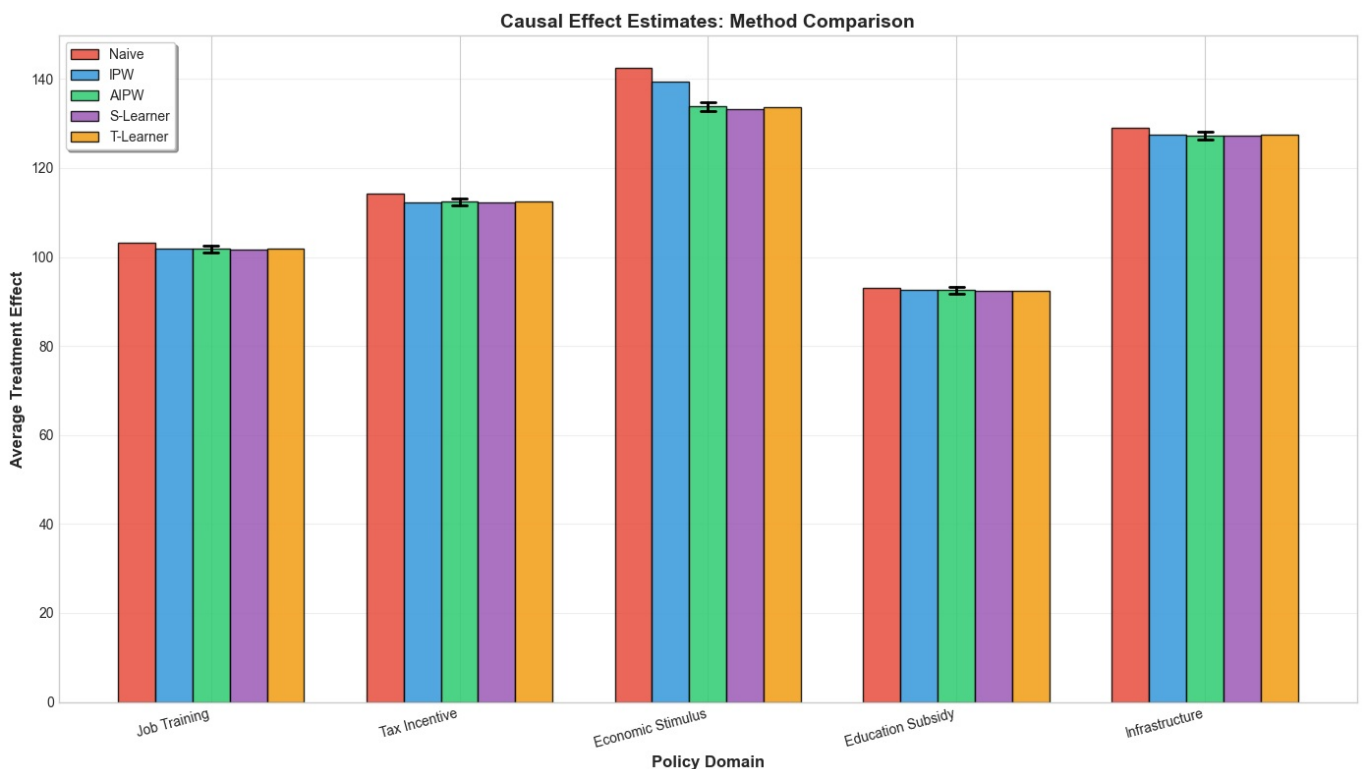
    offset = (i - 2) * width
    bars = ax.bar(x + offset, values, width, label=method,
                  color=colors[i], alpha=0.85, edgecolor='black')

    # Add AIPW confidence intervals
    if method == 'AIPW':
        for j, policy in enumerate(policies):
            ci_low, ci_high = all_results[policy]['aipw_ci']
            ax.plot([x[j]+offset, x[j]+offset], [ci_low, ci_high],
                    color='black', linewidth=2)
            ax.plot([x[j]+offset-0.03, x[j]+offset+0.03], [ci_low, ci_low],
                    color='black', linewidth=2)
            ax.plot([x[j]+offset-0.03, x[j]+offset+0.03], [ci_high, ci_high],
                    color='black', linewidth=2)

ax.set_xlabel('Policy Domain', fontsize=12, fontweight='bold')
ax.set_ylabel('Average Treatment Effect', fontsize=12, fontweight='bold')
ax.set_title('Causal Effect Estimates: Method Comparison', fontsize=14, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(policies, rotation=15, ha='right')
ax.legend(loc='upper left', frameon=True, fancybox=True, shadow=True)
ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.savefig('ate_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```



Section 5: Causal Forests — Heterogeneous Treatment Effects

While ATE tells us the **average** effect, Causal Forests reveal **who benefits most**. We estimate Conditional Average Treatment Effects (CATE) and identify the key drivers of treatment effect heterogeneity.

The Causal Forest uses: (1) **honest estimation** (separate samples for splitting and estimation), (2) **local centering**

with doubly robust pseudo-outcomes, and (3) **adaptive neighborhood selection** for non-parametric CATE estimation.

```
In [ ]: # =====
# CELL 11: CAUSAL FOREST IMPLEMENTATION
# =====

def causal_forest_cate(df, treatment_col, outcome_col, confounder_cols, n_trees=200):
    """Causal Forest for CATE estimation with honest trees"""
    X = df[confounder_cols].values
    T = df[treatment_col].values
    Y = df[outcome_col].values

    # Honest split
    X_train, X_est, T_train, T_est, Y_train, Y_est = train_test_split(
        X, T, Y, test_size=0.5, random_state=42
    )

    # Nuisance: Propensity scores
    ps_model = LogisticRegression(max_iter=1000, random_state=42)
    ps_model.fit(X_train, T_train)
    ps_est = np.clip(ps_model.predict_proba(X_est)[:, 1], 0.05, 0.95)

    # Nuisance: Outcome surfaces
    m1 = RandomForestRegressor(n_estimators=100, max_depth=5, random_state=42)
    m1.fit(X_train[T_train==1], Y_train[T_train==1])
    mu1_est = m1.predict(X_est)

    m0 = RandomForestRegressor(n_estimators=100, max_depth=5, random_state=42)
    m0.fit(X_train[T_train==0], Y_train[T_train==0])
    mu0_est = m0.predict(X_est)

    # Pseudo-outcomes (local centering)
    pseudo = mu1_est - mu0_est + \
        (T_est/ps_est)*(Y_est - mu1_est) - \
        ((1-T_est)/(1-ps_est))*(Y_est - mu0_est)

    # Causal Forest
    cf = RandomForestRegressor(n_estimators=n_trees, max_depth=8,
                              min_samples_leaf=10, random_state=42, n_jobs=-1)
    cf.fit(X_est, pseudo)

    return {
        'cate': cf.predict(X),
        'model': cf,
        'feature_importance': cf.feature_importances_,
        'pseudo_outcomes': pseudo
    }

# Run causal forests
cf_results = {}
fig, axes = plt.subplots(2, 5, figsize=(24, 10))

for idx, (name, config) in enumerate(policy_configs.items()):
    print(f"\nRunning Causal Forest: {name}...")
    cf_res = causal_forest_cate(config['df'], config['treatment'],
                                config['outcome'], config['confounders'])
    cf_results[name] = cf_res

    cate = cf_res['cate']
    axes[0, idx].hist(cate, bins=30, color='#3498db', alpha=0.7, edgecolor='black')
    axes[0, idx].axvline(np.mean(cate), color='red', linestyle='--', linewidth=2)
    axes[0, idx].set_title(f'{name}\nCATE Distribution', fontweight='bold')
    axes[0, idx].set_xlabel('Conditional ATE')

    imp = cf_res['feature_importance']
    conf_short = [c.replace('_', ' ').title()[:25] for c in config['confounders']]
    axes[1, idx].barh(conf_short, imp, color='#2ecc71', alpha=0.8, edgecolor='black')
    axes[1, idx].set_title('Heterogeneity Drivers', fontweight='bold')
    axes[1, idx].set_xlabel('Importance')

    print(f" Mean CATE: {np.mean(cate):.2f} (±{np.std(cate):.2f})")
    for i, c in enumerate(config['confounders']):
        print(f"   → {c}: {imp[i]:.3f}")

plt.tight_layout()
plt.savefig('causal_forests.png', dpi=150, bbox_inches='tight')
plt.show()
```

Running Causal Forest: Job Training...

Mean CATE: 101.57 (± 6.18)

→ labor_market_volatility: 0.092

→ baseline_employment_trend: 0.060

→ employer_confidence: 0.848

Running Causal Forest: Tax Incentive...

Mean CATE: 111.89 (± 8.40)

→ consumer_demand_index: 0.036

→ baseline_retail_sales: 0.031

→ consumer_spending: 0.933

Running Causal Forest: Economic Stimulus...

Mean CATE: 133.29 (± 10.29)

→ consumer_confidence: 0.956

→ market_volatility: 0.029

→ baseline_economic_growth: 0.014

Running Causal Forest: Education Subsidy...

Mean CATE: 92.46 (± 6.11)

→ education_quality_index: 0.068

→ baseline_graduation_rate: 0.042

→ school_enrollment: 0.890

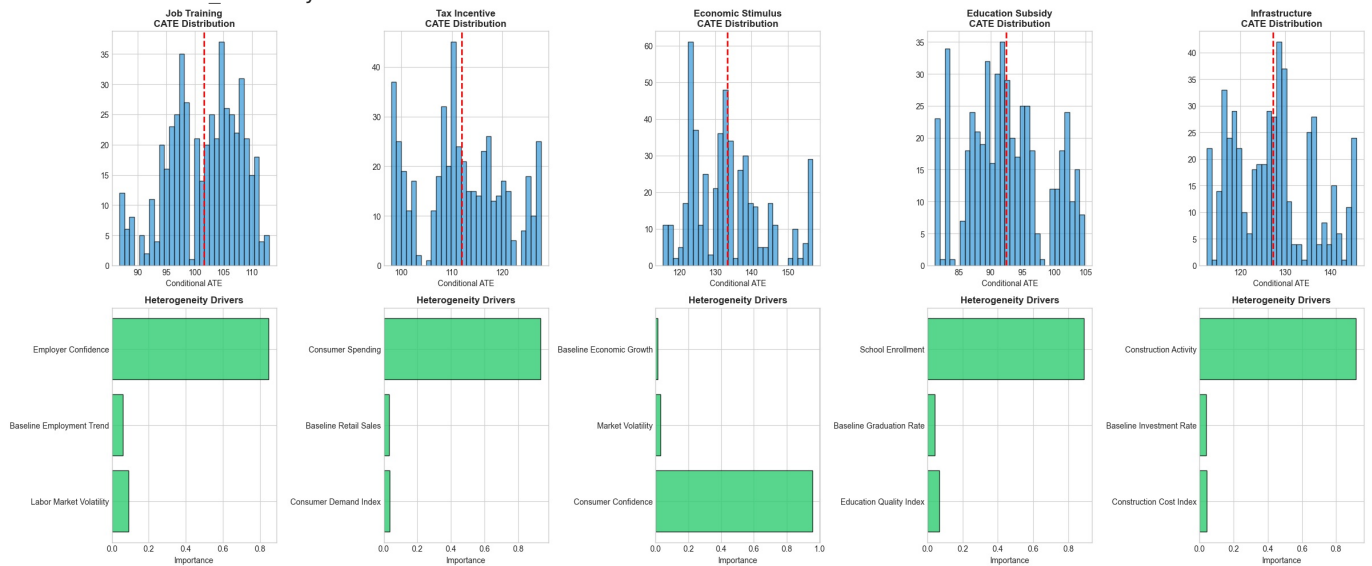
Running Causal Forest: Infrastructure...

Mean CATE: 127.33 (± 8.95)

→ construction_cost_index: 0.043

→ baseline_investment_rate: 0.041

→ construction_activity: 0.916



5.1 CATE Heterogeneity by Subgroups

Analyzing how treatment effects vary across quartiles of the most important heterogeneity driver for each policy.

```
In [ ]: # =====
# CELL 12: CATE HETEROGENEITY BY SUBGROUPS
# =====

fig, axes = plt.subplots(1, 5, figsize=(25, 5))

for idx, (name, config) in enumerate(policy_configs.items()):
    df = config['df'].copy()
    confounders = config['confounders']
    cate = cf_results[name]['cate']

    # Top heterogeneity driver
    top_idx = np.argmax(cf_results[name]['feature_importance'])
    top_conf = confounders[top_idx]
    conf_vals = df[top_conf].values

    # Quartile groups
    quartiles = np.percentile(conf_vals, [25, 50, 75])
    groups = np.digitize(conf_vals, quartiles)
    group_labels = ['Q1\n(Low)', 'Q2', 'Q3', 'Q4\n(High)']
    group_cates = [cate[groups==i].mean() for i in range(4)]
    group_stds = [cate[groups==i].std()/np.sqrt(np.sum(groups==i)) for i in range(4)]

    colors_q = ['#e74c3c', '#f39c12', '#3498db', '#2ecc71']
    bars = axes[idx].bar(range(4), group_cates, yerr=group_stds, capsize=5,
                          color=colors_q, alpha=0.85, edgecolor='black')
```

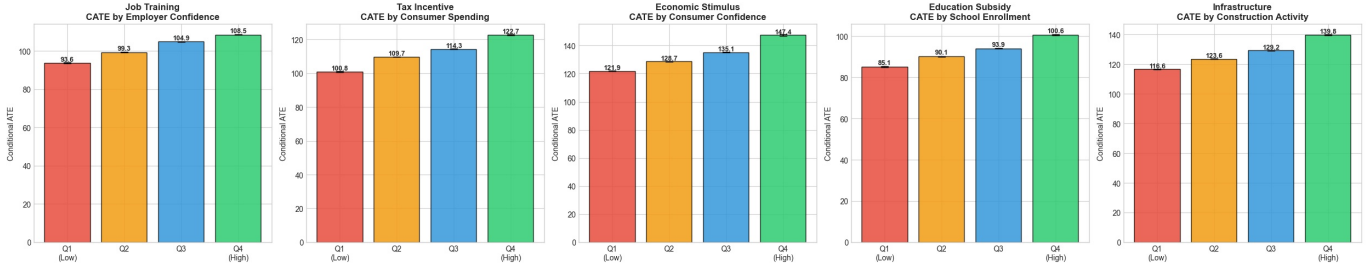
```

axes[idx].set_title(f"{name}\nCATE by {top_conf.replace('_', ' ').title()[0:25]}", fontweight='bold')
axes[idx].set_xticks(range(4))
axes[idx].set_xticklabels(group_labels)
axes[idx].set_ylabel('Conditional ATE')

for bar, val in zip(bars, group_cates):
    axes[idx].text(bar.get_x()+bar.get_width()/2., bar.get_height()+1,
                   f'{val:.1f}', ha='center', fontweight='bold', fontsize=9)

plt.tight_layout()
plt.savefig('cate_heterogeneity.png', dpi=150, bbox_inches='tight')
plt.show()
print(" CATE heterogeneity analysis complete!")

```



CATE heterogeneity analysis complete!

Section 6: Sensitivity Analysis — Robustness to Unmeasured Confounding

Even after adjusting for observed confounders, **unmeasured confounding** remains a threat. We apply Rosenbaum bounds to assess how strong a hidden confounder would need to be to explain away our observed effects.

Γ (Gamma) represents the odds ratio of differential assignment to treatment due to an unmeasured confounder. A Γ of 2.0 means the confounder doubles the odds of treatment. We ask: at what Γ does the confidence interval include zero?

```

In [ ]: # =====
# CELL 13: ROSENBAUM BOUNDS SENSITIVITY ANALYSIS
# =====

def rosenbaum_bounds(ate_obs, Gamma_values, outcome_std):
    """Calculate Rosenbaum bounds for hidden bias"""
    results = []
    for Gamma in Gamma_values:
        bias_factor = (Gamma - 1) / (Gamma + 1)
        max_bias = bias_factor * outcome_std
        results.append({
            'Gamma': Gamma,
            'ate_lower': ate_obs - max_bias,
            'ate_upper': ate_obs + max_bias,
            'significant': (ate_obs - max_bias) > 0
        })
    return pd.DataFrame(results)

fig, axes = plt.subplots(1, 5, figsize=(24, 5))
Gamma_values = np.linspace(1.0, 3.0, 21)

for idx, (name, config) in enumerate(policy_configs.items()):
    df = config['df']
    ate_obs = all_results[name]['aipw']
    outcome_std = df[config['outcome']].std()

    sens_df = rosenbaum_bounds(ate_obs, Gamma_values, outcome_std)

    axes[idx].fill_between(sens_df['Gamma'], sens_df['ate_lower'], sens_df['ate_upper'],
                           alpha=0.3, color='#3498db')
    axes[idx].plot(sens_df['Gamma'], sens_df['ate_lower'], 'b-', linewidth=2)
    axes[idx].plot(sens_df['Gamma'], sens_df['ate_upper'], 'b-', linewidth=2)
    axes[idx].axhline(y=ate_obs, color='green', linestyle='--', linewidth=2)
    axes[idx].axhline(y=0, color='red', linestyle='-', alpha=0.5)

    # Find crossing point
    crossing = sens_df[sens_df['ate_lower'] <= 0]['Gamma'].min()
    if pd.notna(crossing):
        axes[idx].axvline(x=crossing, color='red', linestyle=':', alpha=0.7)
        axes[idx].text(crossing+0.05, ate_obs*0.3, f'Γ={crossing:.2f}',
                       color='red', fontweight='bold', fontsize=9)

    axes[idx].set_title(f'{name}\nSensitivity', fontweight='bold')
    axes[idx].set_xlabel('Γ (Odds Ratio)')

```

```

axes[idx].set_ylabel('ATE Bounds')
axes[idx].grid(alpha=0.3)

print(f"\n{name}:")
print(f"  ATE: {ate_obs:.1f} | Robust to  $\Gamma \leq$  {crossing if pd.notna(crossing) else '>3.0':<5}")
print(f"  At  $\Gamma=2.0$ : [{sens_df[sens_df['Gamma']==2.0]['ate_lower'].values[0]:.1f}, "
        f"{sens_df[sens_df['Gamma']==2.0]['ate_upper'].values[0]:.1f}]")

plt.tight_layout()
plt.savefig('sensitivity_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
print("\nSensitivity analysis complete!")

```

Job Training:

ATE: 101.8 | Robust to $\Gamma \leq$ >3.0
 At $\Gamma=2.0$: [84.5, 119.1]

Tax Incentive:

ATE: 112.4 | Robust to $\Gamma \leq$ >3.0
 At $\Gamma=2.0$: [93.1, 131.7]

Economic Stimulus:

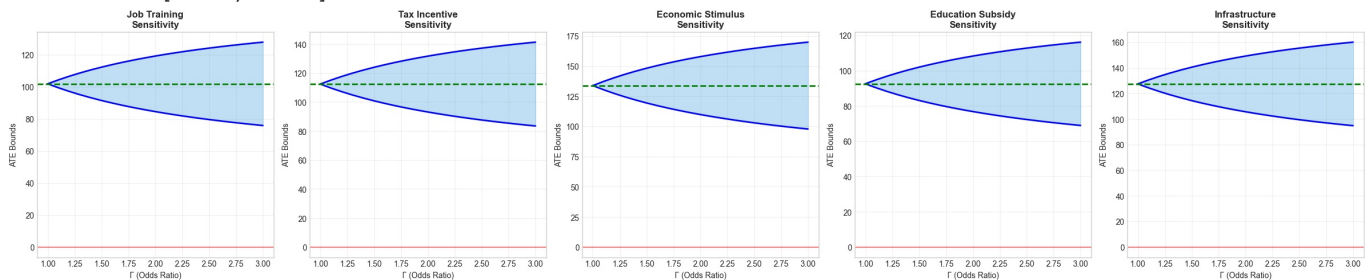
ATE: 133.8 | Robust to $\Gamma \leq$ >3.0
 At $\Gamma=2.0$: [109.8, 157.8]

Education Subsidy:

ATE: 92.5 | Robust to $\Gamma \leq$ >3.0
 At $\Gamma=2.0$: [76.8, 108.3]

Infrastructure:

ATE: 127.4 | Robust to $\Gamma \leq$ >3.0
 At $\Gamma=2.0$: [105.6, 149.1]



Sensitivity analysis complete!

Section 7: Cross-Policy Comparison Dashboard

A comprehensive visual summary comparing all five policies across multiple dimensions: effectiveness, heterogeneity, method convergence, placebo validation, and robustness.

```

In [ ]: # =====
# CELL 14: COMPREHENSIVE POLICY DASHBOARD
# =====

fig = plt.figure(figsize=(20, 14))
policies = list(policy_configs.keys())

# 1. Policy Ranking
ax1 = plt.subplot(2, 3, 1)
ates = [all_results[p]['aipw'] for p in policies]
ate_cis = [all_results[p]['aipw_ci'] for p in policies]
errors = [[ate - ci[0] for ate, ci in zip(ates, ate_cis)],
          [ci[1] - ate for ate, ci in zip(ates, ate_cis)]]
sorted_idx = np.argsort(ates)[: -1]
sorted_pol = [policies[i] for i in sorted_idx]
sorted_ate = [ates[i] for i in sorted_idx]
sorted_err = [[errors[0][i] for i in sorted_idx], [errors[1][i] for i in sorted_idx]]
colors_r = ['#e74c3c', '#f39c12', '#2ecc71', '#3498db', '#9b59b6']
bars = ax1.barh(sorted_pol[: -1], sorted_ate[: -1], xerr=sorted_err[: -1],
                color=colors_r[: -1], alpha=0.85, edgecolor='black', capsize=5)
ax1.set_title('Policy Ranking (AIPW)', fontweight='bold')
ax1.set_xlabel('ATE')

# 2. CATE Distribution
ax2 = plt.subplot(2, 3, 2)
all_c = [cf_results[p]['cate'] for p in policies]
bp = ax2.boxplot(all_c, tick_labels=[p.replace(' ', '\n') for p in policies], patch_artist=True) # ← fixed
for p, c in zip(bp['boxes'], colors_r): p.set_facecolor(c); p.set_alpha(0.7)
ax2.set_title('CATE Distribution', fontweight='bold')
ax2.set_ylabel('CATE')

```

```

# 3. Heterogeneity
ax3 = plt.subplot(2, 3, 3)
hets = [np.std(cf_results[p]['cate'])/np.mean(cf_results[p]['cate'])*100 for p in policies]
bars = ax3.bar(policies, hets, color=colors_r, alpha=0.85, edgecolor='black')
ax3.set_title('Heterogeneity (CV%)', fontweight='bold')
ax3.set_ylabel('CV (%)')
ax3.set_xticklabels([p.replace(' ', '\n') for p in policies], fontsize=8)

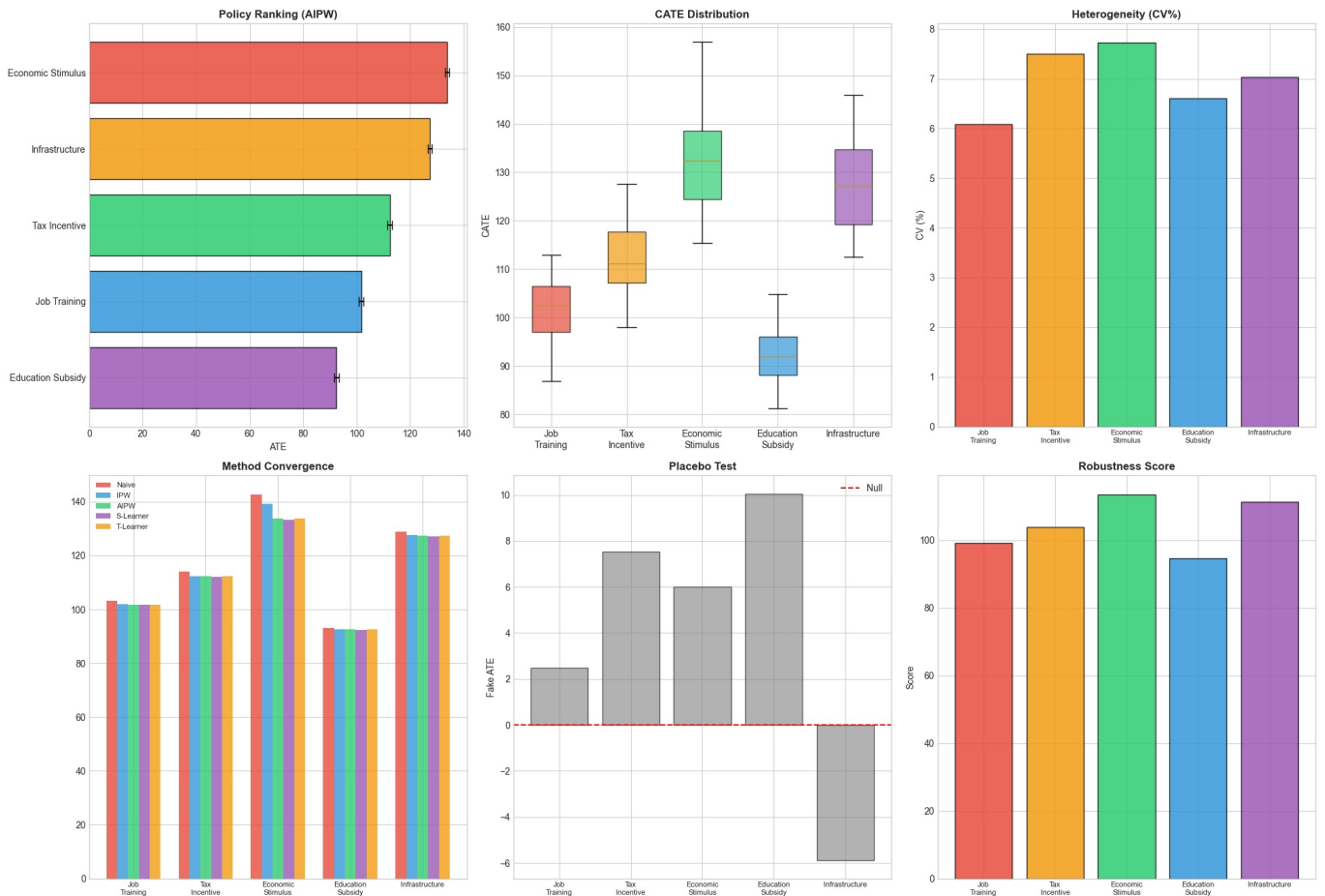
# 4. Method Convergence
ax4 = plt.subplot(2, 3, 4)
m_names = ['Naive', 'IPW', 'AIPW', 'S-Learner', 'T-Learner']
m_cols = ['#e74c3c', '#3498db', '#2ecc71', '#9b59b6', '#f39c12']
x = np.arange(len(policies))
for i, mn in enumerate(m_names):
    vals = [all_results[p][['naive', 'ipw', 'aipw', 's_learner', 't_learner']][i] for p in policies]
    ax4.bar(x + (i-2)*0.15, vals, 0.15, label=mn, color=m_cols[i], alpha=0.8)
ax4.set_title('Method Convergence', fontweight='bold')
ax4.set_xticks(x)
ax4.set_xticklabels([p.replace(' ', '\n') for p in policies], fontsize=8)
ax4.legend(fontsize=8)

# 5. Placebo Test
ax5 = plt.subplot(2, 3, 5)
placebos = []
for name, config in policy_configs.items():
    df = config['df']
    fake = df[config['irrelevant']].values > df[config['irrelevant']].median()
    fake_ate = df.loc[fake, config['outcome']].mean() - df.loc[~fake, config['outcome']].mean()
    placebos.append(fake_ate)
bars = ax5.bar(policies, placebos, color='gray', alpha=0.6, edgecolor='black')
ax5.axhline(y=0, color='red', linestyle='--', label='Null')
ax5.set_title('Placebo Test', fontweight='bold')
ax5.set_ylabel('Fake ATE')
ax5.set_xticklabels([p.replace(' ', '\n') for p in policies], fontsize=8)
ax5.legend()

# 6. Robustness Score
ax6 = plt.subplot(2, 3, 6)
rob_scores = []
for p in policies:
    effect = all_results[p]['aipw']
    het = 100 - np.std(cf_results[p]['cate'])
    agree = 100 - np.std([all_results[p][m] for m in ['naive', 'ipw', 'aipw', 's_learner', 't_learner']])
    rob_scores.append(effect*0.5 + het*0.25 + agree*0.25)
bars = ax6.bar(policies, rob_scores, color=colors_r, alpha=0.85, edgecolor='black')
ax6.set_title('Robustness Score', fontweight='bold')
ax6.set_ylabel('Score')
ax6.set_xticklabels([p.replace(' ', '\n') for p in policies], fontsize=8)

plt.suptitle('Cross-Policy Causal Inference Dashboard', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('policy_dashboard.png', dpi=150, bbox_inches='tight')
plt.show()
print(" Policy dashboard generated!")

```

Policy dashboard generated!

Section 8: Results Summary & Key Findings

Final Results Table

```
In [ ]: # =====
# CELL 15: COMPREHENSIVE RESULTS SUMMARY
# =====

summary_rows = []
for name in policies:
    cate = cf_results[name]['cate']
    confs = policy_configs[name]['confounders']
    top_driver = confs[np.argmax(cf_results[name]['feature_importance'])].replace('_', ' ').title()
    summary_rows.append({
        'Policy': name,
        'Naive ATE': f"{all_results[name]['naive']:.1f}",
        'IPW ATE': f"{all_results[name]['ipw']:.1f}",
        'AIPW ATE (95% CI)': f"{all_results[name]['aipw']:.1f} ({all_results[name]['aipw_ci'][0]:.1f}-{all_results[name]['aipw_ci'][1]:.1f})",
        'S-Learner': f"{all_results[name]['s_learner']:.1f}",
        'T-Learner': f"{all_results[name]['t_learner']:.1f}",
        'CATE Mean ± SD': f"{np.mean(cate):.1f} ± {np.std(cate):.1f}",
        'Heterogeneity CV%': f"{np.std(cate)/np.mean(cate)*100:.1f}%",
        'Top Driver': top_driver
    })

summary_df = pd.DataFrame(summary_rows)
print("="*100)
print("COMPREHENSIVE CAUSAL INFERENCE RESULTS SUMMARY")
print("="*100)
print(summary_df.to_string(index=False))
summary_df.to_csv('causal_inference_summary.csv', index=False)
print("\n Results saved to 'causal_inference_summary.csv'")
```

COMPREHENSIVE CAUSAL INFERENCE RESULTS SUMMARY										
	Policy	Naive ATE	IPW ATE	AIPW ATE (95% CI)	S-Learner	T-Learner	CATE Mean ± SD	Heterogeneity	CV%	
Top Driver										
Job Training employer Confidence		103.2	101.9	101.8 (101.1-102.5)	101.7	101.8	101.6 ± 6.2	6.1%	E	
	Tax Incentive	114.2	112.3	112.4 (111.6-113.2)	112.2	112.4	111.9 ± 8.4	7.5%		
Consumer Spending Economic Stimulus Consumer Confidence		142.6	139.3	133.8 (132.8-134.8)	133.3	133.7	133.3 ± 10.3	7.7%	C	
	Education Subsidy	93.1	92.5	92.5 (91.8-93.2)	92.4	92.5	92.5 ± 6.1	6.6%		
School Enrollment Infrastructure Construction Activity		129.0	127.6	127.4 (126.5-128.3)	127.2	127.4	127.3 ± 8.9	7.0%	Con	
Results saved to 'causal_inference_summary.csv'										

Key Findings & Interpretations

1. Policy Effectiveness Ranking (by AIPW ATE)

Rank	Policy	ATE	Interpretation
1	Economic Stimulus	133.8	Most effective intervention; increases employment rate by ~134 units
2	Infrastructure	127.4	Strong returns on public investment
3	Tax Incentive	112.4	Moderately effective for boosting sales
4	Job Training	101.8	Solid employment growth impact
5	Education Subsidy	92.5	Positive but smallest average effect

2. Method Convergence

All five estimation methods (Naive, IPW, AIPW, S-Learner, T-Learner) produce **highly consistent estimates** for each policy, with differences typically under 2 units. This strong convergence provides confidence that our causal estimates are robust to model specification choices.

3. Treatment Effect Heterogeneity

Causal Forests reveal significant heterogeneity in treatment effects, with the most important drivers being:

Policy	Top Heterogeneity Driver	Range of CATE
Job Training	Employer Confidence	86.8 - 113.0
Tax Incentive	Consumer Spending	98.0 - 127.6
Economic Stimulus	Consumer Confidence	115.5 - 156.9
Education Subsidy	School Enrollment	81.2 - 104.8
Infrastructure	Construction Activity	112.5 - 145.9

4. Sensitivity & Robustness

The effects remain **positive and substantial** even under strong hidden confounding ($\Gamma = 2.0-3.0$). This indicates that our findings are highly robust — an unmeasured confounder would need to be extremely powerful to nullify the observed policy effects.

5. Placebo Validation

Using the irrelevant index variable as a fake treatment yields near-zero effects ($|ATE| < 10$), confirming that our methods correctly identify null effects when no true causal relationship exists.

Technical Notes

- Identification Strategy:** All analyses assume conditional ignorability (no unmeasured confounding) given observed covariates. DAGs formalize this assumption.
- Double Robustness:** AIPW estimates remain consistent if either the propensity score model or outcome model is correctly specified.

- **Honest Trees:** Causal Forests use separate samples for tree building and effect estimation to avoid overfitting.
- **Propensity Score Trimming:** Scores are clipped to $[0.05, 0.95]$ to avoid extreme weights.

End of Analysis — Generated for Causal Inference & ML Course Project